Pandas for Data Science

Complete Reference Notes

Topics Covered

Quick Import

```
import pandas as pd
import numpy as np
```

Black & White Print Edition — All examples compiled from course notebooks
Includes Bonus Section: Time Series, MultiIndex, Performance Tips

Contents

1 Getting Started with Pandas

1.1 What is Pandas?

Pandas is a powerful, open-source Python library for data manipulation, cleaning, and analysis. It is built on top of NumPy and provides two main data structures:

- **Series** — 1D labeled array (like a column in Excel)
- **DataFrame** — 2D labeled table (like a full spreadsheet or SQL table)

Why Pandas over plain Python?

draculaCurrentLine**Task**	**Without Pandas**	**With Pandas**
Load a CSV	`open()` + loops	`pd.read_csv()`
Filter rows	Custom loop logic	`df[df["col"] > 5]`
Group & summarize	Manual aggregation	`df.groupby()`
Merge two datasets	Nested loops	`pd.merge()`

1.2 Installation & Import

```
# Install
pip install pandas              # via pip
conda install pandas            # via Anaconda/conda

# Standard import (always use pd alias)
import pandas as pd
import numpy as np              # usually needed alongside pandas
```

Pandas vs Other Tools

draculaCurrentLine**Tool**	**Strengths**	**Weaknesses**
Excel	Easy UI, good for small data	Slow, manual, not scalable
SQL	Efficient for big data queries	Hard for transformation logic
NumPy	Fast, low-level array ops	No labels, harder for tabular
Pandas	Label-aware, fast, flexible	Slight learning curve

2 Core Data Structures

2.1 Series — 1D Labeled Array

A Series is like a Python list but with labels (index) attached to every element.

```
import pandas as pd

# Auto-indexed (0, 1, 2, ...)
s1 = pd.Series([71, 84, 56, 98, 33])
print(s1)
# 0    71
# 1    84
# 2    56
# 3    98
```

```
# 4    33
# dtype: int64

# Custom index (names instead of numbers)
s2 = pd.Series([71, 84, 56, 98, 33],
               index=["Aritra", "Anshumaan", "Niya", "Kabir", "Ayan"])

# Access by label
print(s2.Kabir)        # 98
print(s2.Anshumaan)    # 84
print(s2["Niya"])      # 56 (bracket notation also works)
```

Series vs Dictionary

Both store key-value pairs, but Series is much more powerful:

- Vectorized operations: `s * 2`, `s + 10` apply to all elements instantly
- Automatic index alignment: arithmetic between two Series aligns on index
- Missing data: handles NaN gracefully
- Position-based access: `s.iloc[0]` works even with custom labels
- Integrates with DataFrames: each column is a Series

2.2 DataFrame — 2D Labeled Table

A DataFrame is a dictionary of Series — multiple columns with labels sharing the same index.

```
data = {
    "name": ["Kabir", "Anshumaan", "Aritra"],
    "age": [11, 22, 33],
    "city": ["Delhi", "Chennai", "Agra"]
}

df = pd.DataFrame(data)
print(df)
#         name  age     city
# 0      Kabir   11    Delhi
# 1  Anshumaan   22  Chennai
# 2     Aritra   33     Agra

# Access structural info
print(df.index)     # RangeIndex(start=0, stop=3, step=1)
print(df.columns)   # Index(['name', 'age', 'city'], dtype='object')
print(df.shape)     # (3, 3) --> (rows, columns)
```

Index & Labels

Every Series and DataFrame has an Index. It enables: fast lookups, data alignment, merging/joining, and time series operations.

```
df.index = ["a", "b", "c"]              # Change row labels
df.columns = ["Name", "Age", "City"]    # Change column labels
```

3 Creating DataFrames

3.1 From Python List of Lists

```
data = [['Kabir', 99], ['Anshumaan', 98], ['Aritra', 77], ['Manan', 88]]
df = pd.DataFrame(data, columns=['Name', 'Marks'])
```

3.2 From Dictionary of Lists (Most Common)

```
data = {"a": [4, 6, 3], "b": [44, 56, 55]}
df = pd.DataFrame(data)
# Each key becomes a column, each list is the column data
```

3.3 From NumPy Array

```
import numpy as np
arr = np.array([[1, 2], [3, 4]])
df = pd.DataFrame(arr, columns=["A", "B"])   # Always provide column names!
```

3.4 From CSV File

```
df = pd.read_csv("data.csv")

# Useful options:
df = pd.read_csv("data.csv", usecols=["Name", "Age"])  # Load specific columns
df = pd.read_csv("data.csv", nrows=100)                # Load only 100 rows
df = pd.read_csv("data.csv", sep=";")                  # Custom separator
df = pd.read_csv("data.csv", index_col="ID")           # Set column as index
df = pd.read_csv("data.csv", header=None,              # No header in file
                  names=["Col1", "Col2"])
```

3.5 From Excel File

```
# Install openpyxl first: pip install openpyxl
df = pd.read_excel("data.xlsx")
df = pd.read_excel("data.xlsx", sheet_name="Sales")    # Specific sheet
```

3.6 From JSON

```
df = pd.read_json("data.json")        # From file
df = pd.read_json("https://...")      # From URL

# Nested JSON (flatten it):
import json
data = [
    {"id": 1, "name": "Alice", "address": {"city": "Delhi", "zip": "110001"}},
    {"id": 2, "name": "Bob", "address": {"city": "Mumbai", "zip": "400001"}}
]
```

```
df = pd.json_normalize(data)    # Flattens nested dicts
# address.city and address.zip become columns
```

3.7 From URL

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
df = pd.read_csv(url)
```

3.8 From SQL Database

```
import sqlite3
conn = sqlite3.connect("mydb.sqlite")
df = pd.read_sql("SELECT * FROM users", conn)
```

3.9 EDA — First Look at Any Dataset

After loading data, always run these first:

```
df.head()         # First 5 rows (df.head(10) for first 10)
df.tail()         # Last 5 rows
df.info()         # Column names, dtypes, non-null counts, memory usage
df.describe()     # Stats: count, mean, std, min, max, quartiles (numeric cols)
df.columns        # List of all column names
df.shape          # (rows, columns) as a tuple
df.dtypes         # Data type of each column
df.nunique()      # Number of unique values per column
df.value_counts()# Frequency count (on a Series)
```

EDA Workflow Tip

Always run `df.info()` + `df.isnull().sum()` immediately after loading. This reveals missing data and wrong dtypes before you start any analysis.

4 Data Selection & Filtering

4.1 Selecting Columns

```
df["Actor"]               # Single column --> returns a Series
df[["Actor", "IMDb"]]     # Multiple columns --> returns a DataFrame

type(df["Actor"])         # pandas.core.series.Series
type(df[["Actor"]])       # pandas.core.frame.DataFrame
```

4.2 Selecting Rows: .loc[] vs .iloc[]

draculaCurrentLine	**.loc[]**	**.iloc[]**
Stands for	Label-based	Integer Position-based
Row access	By index label	By integer position (0, 1, 2...)
Slicing	Inclusive on both ends	Exclusive on right end
Use when	Index has meaningful labels	Index is/was numeric

```
# .loc[] -- label based
df.loc[1]                        # Row with label 1
df.loc[8, "IMDb"]                 # Row label 8, column "IMDb"
df.loc[0:2, "Actor"]              # Rows 0 to 2 (INCLUSIVE), column "Actor"
df.loc[0:2, ["Actor", "IMDb"]]    # Multiple columns

# .iloc[] -- position based
df.iloc[1]             # Row at position 1
df.iloc[8, 5]           # Row at pos 8, column at pos 5
df.iloc[0:3, 0:2]       # Rows 0,1,2 and columns 0,1 (exclusive right)

# Fast single-element access
df.at[0, "Actor"]    # Label-based single cell (faster than .loc for 1 value)
df.iat[0, 0]          # Position-based single cell (faster than .iloc for 1 value)
```

4.3 Boolean Filtering

```
# Single condition
df[df["IMDb"] > 6]

# AND condition (both must be true)
df[(df["IMDb"] > 6) & (df["Year"] > 2018)]

# OR condition (either can be true)
df[(df["IMDb"] > 6) | (df["Year"] > 2018)]

# Negation
df[~(df["Genre"] == "Action")]     # NOT Action

# isin() -- check membership
df[df["Genre"].isin(["Action", "Comedy", "Drama"])]

# between()
df[df["Year"].between(2015, 2020)]

# String matching
df[df["Actor"].str.contains("Khan", case=False)]
```

4.4 Filtering with .query()

`.query()` lets you filter using a readable SQL-like string expression.

```
df.query("Year > 2019")
df.query("Year > 2019 and IMDb > 6")
```

```
# String values in single quotes
df.query("Genre == 'Action'")

# External Python variable -- use @
min_year = 2018
df.query("Year > @min_year")

# Column names with spaces -- use backticks
df.query("`Box Office` > 100")

# Chained comparison
df.query("6 < IMDb <= 8")
```

.query() Rules Summary

- Use `and`, `or`, `not` (NOT `&`, `|`, `~`)
- String values must be in quotes inside the query string
- Use `@variable` to reference external Python variables
- Use backticks for column names with spaces: `` `first name` ``
- Case-sensitive: `"City == 'delhi'"` fails if value is `'Delhi'`
- Returns a copy, not a view — reassign to save changes

4.5 Copy vs View

```
df2 = df.copy()   # Independent copy -- changes to df2 don't affect df

# .query() returns a copy (safe):
filtered = df.query("IMDb > 6")    # filtered is a new DataFrame
```

5 Data Cleaning & Preprocessing

5.1 Handling Missing Values

```
# Detect missing values
df.isnull()                          # DataFrame of True/False (True = missing)
df.isnull().sum()                    # Count of missing values per column
df.notnull()                         # Opposite of isnull()
df.isnull().sum() / len(df) * 100    # % missing per column

# Drop rows/columns with missing values
df.dropna()               # Drop ANY row with at least one NaN
df.dropna(axis=1)         # Drop columns with any NaN
df.dropna(how='all')      # Only drop rows where ALL values are NaN
df.dropna(thresh=3)       # Keep rows with at least 3 non-null values

# Fill missing values
df.fillna(0)                              # Replace all NaN with 0
df["Age"].fillna(df["Age"].mean())        # Fill with column mean
df["City"].fillna("Unknown")              # Fill with a string
df.fillna(method='ffill')   # Forward fill (deprecated; use below)
```

```
df.ffill()                    # Forward fill: copy value from above
df.bfill()                    # Backward fill: copy value from below
```

ffill vs bfill

ffill (forward fill): goes top-to-bottom, fills NaN with the previous value.
bfill (backward fill): goes bottom-to-top, fills NaN with the next value.
Useful for time series data where values should propagate over gaps.

5.2 Detecting & Removing Duplicates

```
df.duplicated()                                    # True for each duplicate row (not the first)
df.duplicated(keep='last')                         # Mark first occurrence as duplicate instead
df.drop_duplicates()                               # Remove duplicate rows (keep first)
df.drop_duplicates(keep='last')                    # Keep last occurrence
df.drop_duplicates(subset=["Name", "Age"])         # Duplicates based on specific columns
```

5.3 String Operations with .str

```
# Vectorized string methods -- always returns a Series
df["Name"].str.lower()                          # Lowercase all names
df["Name"].str.upper()                          # Uppercase
df["Name"].str.strip()                          # Remove leading/trailing spaces
df["Name"].str.replace("Mr.", "")               # Find & replace
df["City"].str.contains("Delhi", case=False)    # Returns True/False Series
df["Email"].str.split("@")                      # Each element becomes a list
df["Name"].str.len()                            # Length of each string
df["Code"].str.startswith("IN")                 # Starts with check
df["Name"].str[:3]                              # Slice first 3 characters

# Chain methods
df["Name"].str.strip().str.lower()              # Clean and lowercase
```

5.4 Type Conversions

```
# .astype() for basic type changes
df["Age"] = df["Age"].astype(int)                       # float -> int (no NaN allowed!)
df["Price"] = df["Price"].astype(float)                 # int -> float
df["Category"] = df["Category"].astype("category")      # Saves memory

# Must drop NaN before int conversion
df2 = df.dropna().copy()
df2["Age"] = df2["Age"].astype(int)

# Check types
df.dtypes

```

```python
# pd.to_datetime() -- for date columns
df["Date"] = pd.to_datetime(df["Date"])                          # Auto-detects format
df["Date"] = pd.to_datetime(df["Date"], format="%d/%m/%Y")       # Explicit format
```

Why pd.to_datetime() instead of astype()?

`pd.to_datetime()` can handle multiple date formats, mixed types, NaT values, UNIX timestamps, and timezones. `astype()` is for simple numeric/string conversions only.

5.5 Applying Functions

```python
# .apply() -- apply any function to rows or columns
df["Age Group"] = df["Age"].apply(lambda x: "Adult" if x >= 18 else "Minor")

# Apply to multiple columns (axis=1 means row-wise)
df["Full Info"] = df.apply(lambda row: f"{row['Name']} from {row['City']}", axis=1)

# .map() -- element-wise mapping for a Series
gender_map = {"M": "Male", "F": "Female"}
df["Gender"] = df["Gender"].map(gender_map)

# .replace() -- replace specific values
df["City"] = df["City"].replace({"Del": "Delhi", "Mum": "Mumbai"})
df["Score"] = df["Score"].replace(-1, None)    # Replace sentinel values
```

draculaCurrentLineMethod	Works on	Best for
`.apply(func)`	Series or DataFrame	Complex custom logic
`.map(dict/func)`	Series only	Element-wise mapping
`.replace()`	Series or DataFrame	Simple value substitution

6 Data Transformation

6.1 Sorting

```python
# Sort by a single column
df.sort_values("Year")                  # Ascending (default)
df.sort_values("Year", ascending=False) # Descending

# Sort by multiple columns (handles ties)
df.sort_values(["Year", "IMDb"])                              # Sort by Year, then IMDb for ties
df.sort_values(["Year", "IMDb"], ascending=[True, False])     # Mixed order

# Sort by index
df.sort_index()     # Restore original index order
```

6.2 Resetting the Index

```python
# After sorting/filtering, the index becomes non-sequential
df2 = df.sort_values(["Year", "IMDb"]).copy()
df2.reset_index()               # Creates new 0-based index, old index as column
df2.reset_index(drop=True)      # Creates new index, discards old index column

# Use inplace=True to modify in place
df2.reset_index(drop=True, inplace=True)   # df2 is updated permanently
```

6.3 Ranking

```python
df["Rank"] = df["IMDb"].rank()                                          # Default: average for ties
df["Rank"] = df["IMDb"].rank(ascending=False)                           # Rank 1 = highest score
df["Rank"] = df["IMDb"].rank(ascending=False, method="dense")           # No gaps in ranks

# method options:
# 'average' -- ties get average rank (default): 1.5, 1.5
# 'min'     -- ties get minimum rank: 1, 1
# 'max'     -- ties get maximum rank: 2, 2
# 'first'   -- rank by order of appearance: 1, 2
# 'dense'   -- like min but no gaps: 1, 2, 2, 3 (not 1, 2, 2, 4)
```

6.4 Renaming Columns & Index

```python
# Rename specific columns
df.rename(columns={"oldName": "newName"}, inplace=True)
df.rename(columns={"Film": "Movie", "IMDb": "Rating"}, inplace=True)

# Rename index labels
df.rename(index={0: "row1", 1: "row2"}, inplace=True)

# Rename all columns at once (list must match column count)
df.columns = ["Name", "Age", "City"]
```

6.5 Reordering Columns

```python
# Specific new order
df = df[["City", "Name", "Age"]]

# Move one column to the front
cols = ["Name"] + [col for col in df.columns if col != "Name"]
df = df[cols]

# Move one column to the end
cols = [col for col in df.columns if col != "Score"] + ["Score"]
df = df[cols]

# Drop a column
df = df.drop(columns=["Rank"])      # Drop "Rank" column
df.drop(columns=["A", "B"], inplace=True)
```

7 Reshaping: Melt & Pivot

7.1 melt() — Wide to Long Format

`melt()` unpivots a DataFrame: turns column names into row values.

```
# Wide format (each subject is a column)
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Math': [85, 78, 92],
    'Science': [90, 82, 89],
    'English': [88, 85, 94]
}
df = pd.DataFrame(data)

# Melt: go from wide to long
df_long = df.melt(
    id_vars=["Name"],                          # Columns to keep as-is
    value_vars=["Math", "Science", "English"], # Columns to melt into rows
    var_name="Subject",                        # Name for the new 'variable' column
    value_name="Score"                          # Name for the new 'value' column
)
```

Result:

draculaCurrentLineName	**Subject**	**Score**
Alice	Math	85
Bob	Math	78
Charlie	Math	92
Alice	Science	90
...	...	...

When to use melt()

Data normalization, preparing data for plotting libraries (matplotlib, seaborn), tidy data format for statistical models.

7.2 pivot() — Long to Wide Format

`pivot()` reverses melt: spreads row values back into columns.

```
# Long format back to wide
df_wide = df_long.pivot(
    index="Name",       # Unique values become row labels
    columns="Subject",  # Unique values become column headers
    values="Score"       # Values to fill the table
)
# Result has Name as index, Math/Science/English as columns
df_wide = df_wide.reset_index()    # Move Name back to a regular column
```

Result:

draculaCurrentLineName	English	Math	Science
Alice	88	85	90
Bob	85	78	82
Charlie	94	92	89

pivot() Important Note

If there are duplicate combinations of (index, columns), `pivot()` raises a `ValueError`. Use `pivot_table()` with an aggregation function instead.

7.3 pivot_table() — Pivot with Aggregation

```
# When there are duplicate entries, use pivot_table()
df.pivot_table(
    index="Name",
    columns="Subject",
    values="Score",
    aggfunc="mean"      # Aggregate duplicates: mean, sum, count, min, max
)

# Multiple aggregations
df.pivot_table(
    index="Name",
    columns="Subject",
    values="Score",
    aggfunc=["mean", "max"]
)
```

draculaCurrentLineWide	Long
Cols: Math, Science	Cols: Subject, Score
Rows: one per student	Rows: one per student-subject

Convert to Long ⇒ `melt()` Convert to Wide ⇐ `pivot()`

8 Aggregation & Grouping

8.1 .groupby() Function

`groupby()` splits the DataFrame into groups, applies a function to each group, and combines results.

Syntax pattern:

`df.groupby(A)[B].C()`

A = how to split — B = which column — C = what calculation

```
import pandas as pd

df = pd.DataFrame({
    "Department": ["HR","HR","IT","IT","Marketing","Marketing","Sales","Sales"],
    "Team": ["A","A","B","B","C","C","D","D"],
    "Gender": ["M","F","M","F","M","F","M","F"],
```

```
    "Salary": [85, 90, 78, 85, 92, 88, 75, 80],
    "Age": [23, 25, 30, 22, 28, 26, 21, 27],
    "JoinDate": pd.to_datetime(["2020-01-10","2020-02-15","2021-03-20","2021-04-10",
                                "2020-05-30","2020-06-25","2021-07-15","2021-08-01"])
})

# Group by one column
df.groupby("Department")["Salary"].mean()     # Average salary per department
df.groupby("Team")["Salary"].sum()              # Total salary per team
df.groupby("Team")["Salary"].count()            # Employee count per team
df.groupby("Team")["Salary"].min()
df.groupby("Team")["Salary"].max()

# Group by multiple columns
df.groupby(["Team", "Gender"])["Salary"].mean()
```

8.2 Custom Aggregations with .agg()

```
# Multiple functions on one column
df.groupby("Team")["Salary"].agg(["mean", "max", "min"])

# Named aggregations
df.groupby("Team")["Salary"].agg(
    avg_score="mean",
    high_score="max"
)

# Different functions for different columns
df.groupby("Team").agg({
    "Salary": "mean",
    "Age": "max"
})
```

.agg and .aggregate

`.agg()` and `.aggregate()` are exactly the same — they are aliases.

8.3 Transform, Aggregate, Filter

draculaCurrentLine**Operation**	**Returns**	**When to Use**
`.aggregate()`	Single value per group	Summary statistics
`.transform()`	Same shape as original	Add a group-based column
`.filter()`	Subset of rows	Keep/discard entire groups

```
# .transform() -- adds group-level value back to each row
df["Team Avg"] = df.groupby("Team")["Salary"].transform("mean")
# Each row now has its team's average salary -- great for comparison

# .filter() -- keep groups meeting a condition (removes entire groups)
df.groupby("Team").filter(lambda x: x["Salary"].mean() > 85)
```

```
# Only keeps teams where the average salary > 85
```

9 Merging & Joining Data

9.1 pd.merge() — SQL-Style Joins

```
employees = pd.DataFrame({
    "EmpID": [1, 2, 3],
    "Name": ["Alice", "Bob", "Charlie"],
    "DeptID": [10, 20, 30]
})

departments = pd.DataFrame({
    "DeptID": [10, 20, 40],
    "DeptName": ["HR", "Engineering", "Marketing"]
})
```

```
# INNER JOIN (default) -- only matching rows
pd.merge(employees, departments, on="DeptID")
# Result: Alice(HR), Bob(Engineering) -- Charlie(30) and Marketing(40) excluded

# LEFT JOIN -- all employees, NaN where no dept match
pd.merge(employees, departments, on="DeptID", how="left")
# Charlie gets NaN for DeptName

# RIGHT JOIN -- all departments, NaN where no employee
pd.merge(employees, departments, on="DeptID", how="right")
# Marketing gets NaN for EmpID/Name

# OUTER JOIN -- everything, NaN for missing
pd.merge(employees, departments, on="DeptID", how="outer")
```

INNER **LEFT** **RIGHT** **OUTER**

9.2 Merging on Different Column Names

```
# When key columns have different names in each DataFrame
pd.merge(df1, df2, left_on="EmpID", right_on="EmployeeID")

# Merge on index
pd.merge(df1, df2, left_index=True, right_index=True)

# .join() method (index-based merge shortcut)
df1.join(df2, how="left")
```

9.3 pd.concat() — Stack DataFrames

```
# Vertical stacking (add more rows)
df1 = pd.DataFrame({"Name": ["Alice", "Bob"], "score": [423, 634]})
df2 = pd.DataFrame({"Name": ["Charlie", "David"], "age": [45, 78]})
pd.concat([df1, df2])      # Stacks rows, aligns columns, NaN for missing
```

```python

# Horizontal stacking (add more columns)
df1 = pd.DataFrame({"ID": [1, 2]})
df2 = pd.DataFrame({"Score": [90, 80]})
pd.concat([df1, df2], axis=1)    # Side by side -- ensure indexes align!

# Reset index after concat
pd.concat([df1, df2], ignore_index=True)   # Fresh 0-based index
```

merge vs concat — When to Use What

draculaCurrentLine**Scenario**	**Use**
SQL-style join (common key column)	`pd.merge()`
Stack more rows (same columns)	`pd.concat([df1, df2])`
Stack more columns (same rows)	`pd.concat([df1, df2], axis=1)`
Join on index	`df.join()` or `pd.merge(..., left_index=True)`

10 Reading & Writing Files

10.1 CSV

```python
# Read
df = pd.read_csv("data.csv")
df = pd.read_csv("data.csv", usecols=["Name", "Age"], nrows=10)
df = pd.read_csv("data.csv", encoding="utf-8")    # Handle encoding issues

# Write
df.to_csv("output.csv", index=False)              # index=False prevents row
    numbers column
df.to_csv("output.csv", columns=["Name", "Age"])   # Only specific columns
```

10.2 Excel

```python
# Read
df = pd.read_excel("data.xlsx")
df = pd.read_excel("data.xlsx", sheet_name="Sales")
df = pd.read_excel("data.xlsx", sheet_name=0)     # First sheet by position

# Write
df.to_excel("output.xlsx", index=False)

# Multiple sheets
with pd.ExcelWriter("report.xlsx") as writer:
    df1.to_excel(writer, sheet_name="Summary", index=False)
    df2.to_excel(writer, sheet_name="Details", index=False)
```

10.3 JSON

```python
df = pd.read_json("data.json")
df.to_json("output.json")
df.to_json("output.json", orient="records")   # List of dicts format
```

11 Bonus Topics: What You Also Need

11.1 Working with Dates & Times

```python
# Parse date columns
df["Date"] = pd.to_datetime(df["Date"])
df["Date"] = pd.to_datetime(df["Date"], format="%d-%m-%Y")

# Extract components
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["Day"] = df["Date"].dt.day
df["Weekday"] = df["Date"].dt.day_name()    # "Monday", "Tuesday"...

# Date arithmetic
df["Days Since"] = pd.Timestamp("today") - df["Date"]
df["Next Month"] = df["Date"] + pd.DateOffset(months=1)

# Filter by date
df[df["Date"] > "2020-01-01"]
df[df["Date"].between("2020-01-01", "2021-12-31")]
```

11.2 Handling Categorical Data

```python
# Convert to category (saves memory, faster operations)
df["Gender"] = df["Gender"].astype("category")
df["Gender"].cat.categories    # See categories
df["Gender"].cat.codes         # Integer codes for each category

# pd.cut() -- bin continuous data into categories
df["Age Group"] = pd.cut(df["Age"], bins=[0, 18, 35, 60, 100],
                         labels=["Child", "Young", "Middle", "Senior"])

# pd.qcut() -- bin by quantiles (equal-sized groups)
df["Salary Tier"] = pd.qcut(df["Salary"], q=4,
                            labels=["Low", "Medium", "High", "Top"])
```

11.3 MultiIndex

```python
# Create MultiIndex
df = df.set_index(["Department", "Team"])

# Access with MultiIndex
df.loc[("HR", "A")]            # Both levels
df.loc["IT"]                   # All rows in IT
df.xs("A", level="Team")       # Cross-section by level

```

```python
# Reset MultiIndex
df = df.reset_index()
```

11.4 Window Functions

```python
# Rolling calculations (moving window)
df["7-day avg"] = df["Sales"].rolling(window=7).mean()
df["Cumulative"] = df["Sales"].cumsum()
df["Running Max"] = df["Score"].cummax()
df["Running Min"] = df["Score"].cummin()

# Expanding window (grows with each row)
df["Expanding Mean"] = df["Sales"].expanding().mean()
```

11.5 Performance Tips

```python
# Avoid chained indexing -- always copy explicitly
df["A"][0] = 5          # BAD -- may give SettingWithCopyWarning
df.at[0, "A"] = 5       # GOOD

# Use vectorized operations, not loops
# BAD (slow):
for i in range(len(df)):
    df.at[i, "New"] = df.at[i, "A"] * 2

# GOOD (fast):
df["New"] = df["A"] * 2

# Check memory usage
df.info(memory_usage="deep")
df.memory_usage(deep=True)

# Reduce memory -- use appropriate types
df["Flag"] = df["Flag"].astype("int8")        # Instead of int64
df["Name"] = df["Name"].astype("category")    # Repeated strings
```

11.6 Useful Utility Functions

```python
# Value counts
df["Genre"].value_counts()                  # Frequency of each value
df["Genre"].value_counts(normalize=True)    # As proportions (%)

# Correlation
df.corr()                   # Pairwise correlation between numeric columns
df["A"].corr(df["B"])       # Correlation between two columns

# Crosstab
pd.crosstab(df["Gender"], df["Department"])   # Frequency table

# Pivot with count
pd.crosstab(df["Gender"], df["Department"], normalize="all")    # % form

```

```python
# Explode (for list columns)
df["Tags"] = df["Tags"].str.split(",")
df_exploded = df.explode("Tags")    # One row per tag

# Sample randomly
df.sample(5)            # 5 random rows
df.sample(frac=0.1)     # 10% of rows

# Clip values
df["Score"] = df["Score"].clip(lower=0, upper=100)    # Cap at 0-100
```

11.7 Cheat Sheet: Most Important Methods

draculaCurrentLine**Load Data**	**Explore**	**Select**
`pd.read_csv()`	`.head(), .tail()`	`df["col"]`
`pd.read_excel()`	`.info(), .describe()`	`.loc[], .iloc[]`
`pd.read_json()`	`.shape, .dtypes`	`.at[], .iat[]`

draculaCurrentLine**Filter**	**Clean**	**Transform**
`df[condition]`	`.isnull(), .fillna()`	`.sort_values()`
`.query("...")`	`.dropna()`	`.rename(), .reset_index()`
`.isin(), .between()`	`.drop_duplicates()`	`.apply(), .map()`
	`.astype(), .str.*`	

draculaCurrentLine**Reshape**	**Aggregate**
`.melt(), .pivot()`	`.groupby().agg()`
`.pivot_table()`	`.transform(), .filter()`
	`pd.concat(), pd.merge()`

End of Notes — Built from Notebooks + Pandas Handbook + Bonus Topics